HONEYPOT-BASED THREAT DETECTION SYSTEM

Capturing & Analyzing Malicious Actor Behavior in an Isolated Environment
Computer Science Senior Project | Instructor: Dr. Pramod Gupta

Rei Thao | Diego Arroyo

Introduction

What is a Honeypot?

A honeypot is a decoy system intentionally configured to attract malicious actors. By simulating vulnerable services, it captures attacker behavior in a controlled, isolated environment without exposing production systems to risk.

Motivation

Cybersecurity threats are constantly evolving. Understanding real attacker behavior is critical for building better defenses. Traditional security tools detect threats after the fact — honeypots provide proactive intelligence by observing attackers in action.

Objectives

- Deploy T-Pot CE multi-honeypot framework on Ubuntu 24.04
- Configure Cowrie SSH/Telnet honeypot with realistic fake environment
- Set up Dionaea for malware capture on common exploit ports
- Implement Canarytokens for credential-use alerting
- Validate detection via simulated Kali Linux attacks
- Visualize and analyze attack data via Kibana dashboards

System Architecture

T-Pot CE Framework

T-Pot CE orchestrates multiple honeypots as Docker containers on a single Ubuntu 24.04 host. All traffic is captured, logged, and forwarded to an Elasticsearch/Kibana stack for analysis.

Components Deployed:

🔒 Cowrie — SSH/Telnet honeypot (Port 22/23)
🦠 Dionaea — Malware capture (SMB, FTP, HTTP)
🪤 Canarytokens — Credential alerting
📊 Kibana — Attack visualization dashboard

Isolation:
System runs in an isolated VMware environment. Port forwarding directs SSH (22/23) to Cowrie while real admin access uses port 64295.

Methodology

Phase 1 — Infrastructure Setup

Installed T-Pot CE on Ubuntu 24.04 in VMware. Configured Docker volume mounts to persist honeypot configuration across container restarts. Real SSH admin access moved to port 64295.

Phase 2 — Cowrie Configuration

Configured Cowrie with custom userdb.txt (UserDB auth), fake filesystem (fs.pickle), and honeyfs file contents. Created four realistic fake users with role-appropriate files:

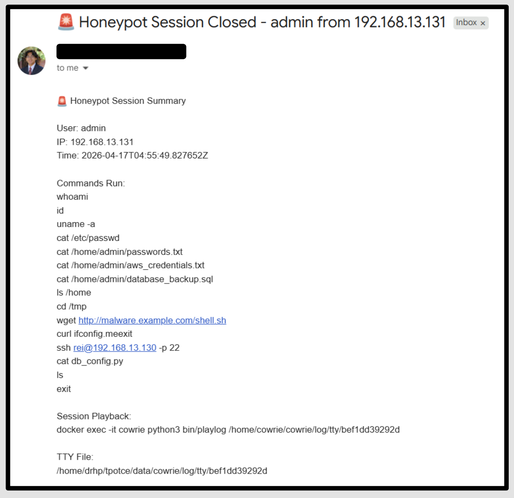
admin → passwords.txt, aws_credentials.txt, database_backup.sql
rei → db_config.py, app.py, notes.txt (w/ staging passwords)
mack → onboarding.txt (w/ temp credentials)
root → server_config.txt, .bash_history

Phase 3 — Canarytoken Integration

Generated three Canarytokens via canarytokens.org:
- AWS Key token embedded in aws_credentials.txt
- Word Document token as confidential.docx
- Web Bug URLs embedded in rei/notes.txt and mack/onboarding.txt

Phase 4 — Alerting Pipeline

Custom Python script monitors Cowrie JSON logs in real time. On session close, email alert sent via Gmail SMTP containing: username, source IP, all commands executed, and TTY session playback command.



Phase 5 — Attack Simulation

Attacks simulated from Kali Linux VM using industry-standard tools to validate full detection pipeline across all three honeypot types.

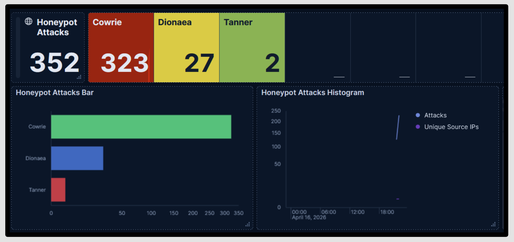
Results

Cowrie SSH Honeypot

- Hydra brute force captured: 270+ login attempts from Kali (192.168.13.131)
- Successful credential matches: admin/admin, rei/reithao, mack/mack123
- Attacker commands recorded: whoami, cat /etc/passwd, cat passwords.txt, cat aws_credentials.txt, wget malware attempts
- Full TTY session playback available for all sessions
- CVE-2024-6387 (OpenSSH vulnerability) probed by scanner

Dionaea Malware Honeypot

- 27 malware capture attempts logged across SMB, FTP, HTTP ports
- EICAR test file successfully captured and stored in binaries directory
- Attacker source IPs geolocated via Kibana attack map



Canarytoken Alerting

- AWS credential token embedded in admin's fake aws_credentials.txt
- Web bug URLs embedded in rei/notes.txt and mack/onboarding.txt
- Word document canarytoken placed as confidential.docx
- Tokens trigger instant email alert when credentials are used or files opened

Email Alerting System

- Custom Python script monitors cowrie.json log in real time
- Alert triggered on session close containing: username, source IP, all commands run, TTY playback command
- Alerts sent via Gmail SMTP with app password authentication

Future Work

Deploy on public internet for real-world attack data. Expand Canarytoken coverage. Integrate SIEM for automated correlation. Add more honeypot types (web, ICS/SCADA). Develop automated threat intelligence reporting.

Conclusion

Successfully deployed a multi-layer honeypot system demonstrating real attacker behavior capture. The system validated detection across SSH brute force, malware uploads, and credential theft scenarios. Email alerting and Kibana visualization provide actionable intelligence for defenders.

Attack Simulation

Tools Used (Kali Linux)

Nmap — Reconnaissance & port scanning
Hydra — SSH brute force credential attacks
Manual SSH — Simulated post-exploitation
curl/wget — Malware download simulation
EICAR test file — Safe malware trigger for Dionaea

Attack Scenario

Stage 1 Reconnaissance: Nmap scan of all honeypot ports
Stage 2 Brute Force: Hydra SSH attack using fasttrack wordlist
Stage 3 Exploitation: Manual SSH as admin, rei, mack
Stage 4 Exfiltration: Cat fake credentials, trigger Canarytokens
Stage 5 Malware: Upload EICAR file via HTTP to Dionaea

Acknowledgements

We would like to thank Shawn Kerns and Dr. Pramod Gupta for their invaluable guidance and support throughout this project. We would also like to acknowledge the T-Pot CE project by Telekom Security for the open-source honeypot framework, as well as the canarytokens.org project.